

M12 / ASSESSMENT

M12 Module Revision Test

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

Closed book - no forced timer 3

1. DE12-01 - Why Spark exists: driver, executors, partitions and tasks 3

2. DE12-02 - PySpark setup, SparkSession and laptop-safe local mode 3

3. DE12-03 - DataFrames and explicit schemas 3

4. DE12-04 - Spark SQL and expression equivalence 3

5. DE12-05 - Lazy transformations and actions 3

6. DE12-06 - Jobs, stages, tasks and physical plans 3

7. DE12-07 - repartition vs coalesce 3

8. DE12-08 - Aggregations and shuffle cost 4

9. DE12-09 - Joins and broadcast strategy 4

10. DE12-10 - Join correctness, fan-out and reconciliation 4

11. DE12-11 - Data skew 4

12. DE12-12 - Adaptive Query Execution (AQE) 4

13. DE12-13 - Caching and persistence 4

14. DE12-14 - Parquet I/O and partition pruning 4

15. DE12-15 - Built-in functions vs Python UDFs 4

16. DE12-16 - Driver safety: collect, toPandas and bounded inspection 4

17. DE12-17 - Testing and debugging Spark transformations 5

18. DE12-18 - Integrated PySpark pipeline and distributed-processing checkpoint 5

Read this first

Closed book - no forced timer

Save the complete attempt before opening protected answers. This test covers all 18 lessons and does not replace a formal Gate (none follows M12).

1. DE12-01 - Why Spark exists: driver, executors, partitions and tasks

Draw driver -> stage -> tasks -> partitions from memory. Explain what changes and what does not change when the same 18-row job moves from local[2] to a real cluster. Name two things that can scale out and two business-correctness rules that must remain identical.

2. DE12-02 - PySpark setup, SparkSession and laptop-safe local mode

Write a one-page learner-PC Spark setup record: Python version, Java version, PySpark version, master, driver-memory setting, shuffle partitions, AQE value and the exact command that successfully created/stopped a session. Mark anything not executed as pending.

3. DE12-03 - DataFrames and explicit schemas

Create an explicit schema for a fresh shipment file with shipment_id, shipped_at, customer_id, pieces and weight_kg. State nullability decisions and one bad-value policy. Then write the DataFrame read skeleton without using inferSchema.

4. DE12-04 - Spark SQL and expression equivalence

Write both DataFrame and Spark SQL versions of Completed gross sales by category. Expected Accessories 16830 and Office 8400. State the result grain and one method to compare results independent of display order.

5. DE12-05 - Lazy transformations and actions

Classify these as transformations, actions or context-dependent APIs: select, filter, withColumn, groupBy result construction, count, first, write.parquet, explain. For each action, state what evidence or side effect it requests.

6. DE12-06 - Jobs, stages, tasks and physical plans

Given read -> filter Completed -> select region, amount -> groupBy region -> sum -> show, mark the likely narrow work and shuffle boundary. State what you would inspect in explain and what only the runtime UI can confirm.

7. DE12-07 - repartition vs coalesce

For a 200 GB output on a cluster, explain what evidence you would collect before choosing a target partition/file count. Include total bytes, current partitions, task duration/skew and downstream read pattern. Do not give one magic number.

8. DE12-08 - Aggregations and shuffle cost

Design an aggregation to compute Completed gross sales by category. State where a shuffle is likely, which columns are needed before it, expected two-row output and the independent 25,230 reconciliation.

9. DE12-09 - Joins and broadcast strategy

Given 2 TB of fact data and a 5 MB static country-code lookup, explain why broadcast is a candidate. List three checks before hinting it and two runtime signs that the choice is unhealthy.

10. DE12-10 - Join correctness, fan-out and reconciliation

Repair the contact join three ways on paper: (a) output one row/customer before joining, (b) preserve order-contact grain and remove order-gross aggregation, (c) use a semi join if only existence matters. State which business question each answers.

11. DE12-11 - Data skew

Design a skew diagnostic checklist using key frequency, partition size, task duration and shuffle bytes. Then explain how you would verify a salting strategy still reconciles to the unsalted total.

12. DE12-12 - Adaptive Query Execution (AQE)

Write a controlled AQE experiment plan: same input and query, AQE true versus false, capture plan, task/partition metrics and exact business output. List confounders you would hold constant.

13. DE12-13 - Caching and persistence

For three cases - single final write, DataFrame reused by six aggregates, and tiny lookup broadcast repeatedly - decide cache/no-cache and state what runtime evidence would support the choice.

14. DE12-14 - Parquet I/O and partition pruning

Design a Parquet layout for two years of daily orders. Compare partitioning by date, region, customer_id and order_id. State the likely query pattern and small-file risk behind your choice.

15. DE12-15 - Built-in functions vs Python UDFs

Review five transformations - uppercase, regex replace, date difference, custom cryptographic library call, complex proprietary scoring function. Mark built-in first versus UDF candidate and explain why.

16. DE12-16 - Driver safety: collect, toPandas and bounded inspection

Rewrite three unsafe patterns: `len(df.collect())`, `df.toPandas().groupby(...)`, and looping over all collected rows to write files. Use distributed Spark equivalents and state any tiny bounded result you still retrieve.

17. DE12-17 - Testing and debugging Spark transformations

Write a test plan for the bad contact join: one assertion should intentionally prove the naive join fails cardinality, and a repaired join should pass unique order count and 25,230 gross reconciliation.

18. DE12-18 - Integrated PySpark pipeline and distributed-processing checkpoint

Build `m12_independent_pipeline.py` from a blank file using the supplied CSVs and common session helper. Required evidence: explicit schema, local-safe session, Completed filter, gross calculation, region+category reconciliation, one explain output, one deliberate partition decision, bounded driver inspection, Parquet write, at least two DataFrame assertions, and a short note explaining any shuffle/broadcast/cache choice you made or deliberately did not make.